CRYPTOGRAPHY · COURSE PROJECT REPORT

Secure Cloud File Sharing Using Hybrid Cryptography

Re-implementation, security analysis and improvement of the Hybrid AES-ECC model of Rehman et al. (Electronics, 2021)

Project type: Research + Application

Group 18

Name	Roll number
Saurav Singh	S20230010219
Rohan Dubey	S20230010207
Ujjawal	S20230010245
Aditya Raj	S20230030364

Demo application: CryptoVault (MongoDB · Express · React · Node.js)

November 2026

Abstract

Hybrid cryptography protects files stored in the cloud by combining a fast symmetric cipher for the data with public-key cryptography for the key. We studied the Hybrid AES-ECC model of Rehman et al. (Electronics 10(21):2673, 2021), which claims a secure and optimized scheme for sharing data over the cloud that ensures authentication and data integrity. Because the authors' repository contains no code, we re-implemented the scheme from the algorithm boxes in the paper, inside a full MERN web application (CryptoVault), and attacked it.

We identify three challenges. **C1:** the AES key is derived from the ECC *public* key, so anyone can decrypt; our attacker recovered 100/100 files using only public information. **C2:** there is no integrity or authentication mechanism and the cipher is used in an ECB-like way; 0 of 1,250 tampering attempts were detected and 98.2% of the 16-byte blocks of an encrypted image repeated. **C3:** the cloud provider performs encryption (it sees plaintext), sharing is only an access list, and revocation is impossible; the paper's evaluation also has methodological errors.

We propose an **Authenticated Hybrid Envelope (AHE)**: a fresh 256-bit random data key per file, AES-256-GCM with chunk-bound associated data, ECIES-style ECDH (P-256) + HKDF key wrapping for every recipient, ECDSA (P-256/SHA-256) signatures over a manifest, client-side (end-to-end) encryption with password-sealed private keys, and revocation by re-keying. Under the same experiments the attacker recovers 0/100 files, 100% of tampering is detected, and the ciphertext reaches 8.00 bits/byte of Shannon entropy with no repeated blocks. The cost is roughly 3 ms per file plus about 390 MB/s throughput on large files. All parts of the solution come from the course syllabus (AES and modes of operation, ECC and Diffie-Hellman, hash functions and digital signatures, Shannon entropy, key management).

Keywords: cloud storage, hybrid cryptography, AES-GCM, ECC, ECDH, ECDSA, authenticated encryption, key management, Shannon entropy.

Contents

Abstract....2

Contents....2

1. Introduction....5

1.1 Introduction to the topic....5

1.2 Background and motivation....5

1.3 Importance in cyber security....5

1.4 Problem statement....5

1.5 Objectives....5

1.6 Threat model and assumptions....6

1.7 Mapping to the course syllabus....6

2. Overview of the Existing Article....6

2.1 The selected article....6

2.2 Technologies used in the paper....7

2.3 Working methodology of the base scheme....7

2.4 Results reported by the paper....7
2.5 Advantages and applications....8
3. Implementation of the Existing Approach....8
3.1 Tools, technologies and languages....8
3.2 Implementation methodology....9
3.3 Experimental setup....9
3.4 Results of the implementation....9
4. Challenges and Limitations....10
4.1 Challenge C1 – The AES key is derived from the public key....10
4.2 Challenge C2 – No integrity or authentication; ECB-style encryption....10
4.3 Challenge C3 – No real sharing, revocation or fair evaluation....11
5. Proposed Approach: Authenticated Hybrid Envelope (AHE)....11
5.1 Selected challenges....11
5.2 Design principles....12
5.3 Building blocks....12
5.4 Algorithms (pseudocode)....13
5.5 Security analysis (informal)....13
5.6 Limitations of the proposed approach....13
6. Implementation of the Proposed Approach....13
6.1 Implementation process and code structure....13
6.2 Application walk-through....15
6.3 Experimental setup (security and performance)....16
7. Results and Analysis....16
7.1 Summary....16
7.2 Attack 1: decryption with public information (C1)....16
7.3 Attack 2: tampering by the cloud (C2)....17
7.4 Pattern leakage, entropy and determinism....18
7.5 Avalanche effect re-evaluated....18
7.6 Performance....19
7.7 Sharing and revocation....20
7.8 Does the proposed approach address the challenges?....20
8. Conclusion....21
9. References....21
Appendix A. Running the demonstration....22
Appendix B. Division of work....22

1. Introduction

1.1 Introduction to the topic

Cloud storage services let users keep files on remote servers and share them with others from any device. The data owner, however, loses physical control: the files sit on infrastructure run by a third party and are reachable by its administrators, its backups and, after a breach, by outsiders. Cryptography is the only protection that does not depend on trusting the provider.

In practice such systems use **hybrid cryptography**. A symmetric cipher such as AES encrypts the (large) file because it is fast, and a public-key algorithm such as RSA or ECC protects or distributes the symmetric key, because public-key cryptography solves the key-distribution problem. The same pattern is used in TLS, PGP and S/MIME.

1.2 Background and motivation

Many papers propose hybrid schemes for cloud storage, and they are usually evaluated mainly on speed (encryption time, key size, power). Speed means little if the construction is insecure: a scheme can use strong primitives (AES, ECC) and still fail because of how they are combined, for example in how keys are derived, which mode of operation is used, or who holds the keys. Our motivation was therefore to take one well-cited published scheme, rebuild it exactly, and **attack** it rather than only time it.

1.3 Importance in cyber security

File sharing must provide the classic security goals: **confidentiality** (only intended users read the file), **integrity** (no undetected modification), **authentication** (the file really comes from the claimed owner), and correct **access control and revocation**. Integrity matters as much as secrecy. A silently modified payment order, contract or medical record is as harmful as a leaked one. These goals are the course objectives (confidentiality, integrity, authentication, non-repudiation).

1.4 Problem statement

Does the published Hybrid AES-ECC model (Rehman et al., 2021) actually keep shared cloud files confidential and unmodified against a curious or malicious cloud and outsiders who obtain the stored data? If not, how can it be fixed, using standard primitives from the course, without making it impractically slow?

1.5 Objectives

1. Re-implement the base scheme faithfully from the paper (its published repository contains no code).
2. Identify at least three challenges/limitations, and demonstrate each with a working attack or measurement.
3. Design an improved scheme that solves at least one challenge. We address all three.
4. Implement both schemes in a working MERN application whose interface shows every cryptographic step live (for the demonstration).
5. Evaluate both schemes with the same security and performance experiments and compare them.

1.6 Threat model and assumptions

- **Cloud provider (honest-but-curious or malicious):** can read everything it stores, can modify, delete, reorder or swap stored bytes, and can serve them to anyone.
- **Outsider with a leaked copy:** obtains the stored ciphertext and metadata, e.g. from a misconfigured bucket or a stolen backup, and can read the public-key directory.
- **Revoked user:** previously had access and must not be able to read versions created after revocation.
- **Assumptions:** the users' own devices/browsers are not compromised, transport is protected by TLS, and users can compare public-key fingerprints out-of-band when needed.

1.7 Mapping to the course syllabus

Table 1. Course topics used in the project.

Course unit / module	Concept	Where it is used in this project
Unit 1 – Number theory	Modular arithmetic, inverses, primes	Prime-field arithmetic behind ECC point operations and ECDSA (k^{-1} mod n)
Unit 2 – Shannon's theory	Entropy, key equivocation, perfect secrecy	C1 analysis H(K | public key) = 0; ciphertext entropy metric; password entropy meter
Module 3 – Symmetric crypto	AES, finite fields, modes (ECB, CBC, CTR)	Base AES-ECB; proposed AES-256-GCM (CTR + GHASH in GF(2^{128})); ECB leakage
Module 4 – Public-key crypto	ECC (and RSA for comparison)	P-256 key pairs; ECC vs RSA key size and speed
Module 5 – Hash & signatures	SHA family, DSA / digital signatures	SHA-256 of ciphertext, HKDF, ECDSA-signed manifest
Unit 6 – Applications	Diffie-Hellman, key distribution and management	ECDH key wrapping per recipient, sharing, revocation by re-keying

2. Overview of the Existing Article

2.1 The selected article

S. Rehman, N. T. Bajwa, M. A. Shah, A. O. Aseeri and A. Anjum, "Hybrid AES-ECC Model for the Security of Data over Cloud Storage", *Electronics*, vol. 10, no. 21, art. 2673, 2021 (MDPI, open access, peer reviewed), DOI 10.3390/electronics10212673. The paper argues that a single algorithm is either slow or insecure, and proposes combining ECC and AES "to ensure authentication and data integrity" for data shared over cloud storage "without involving a third party".

The paper lists three contributions:

- A hybrid model where the key used by AES is generated with the help of ECC, so that the key size is reduced.
- Reduced computational power and memory compared with existing symmetric/asymmetric schemes.
- An algorithm describing how the public key is generated with ECC and how encryption/decryption is done with AES.

2.2 Technologies used in the paper

2.2.1 Advanced Encryption Standard (AES)

AES (FIPS-197) is a symmetric block cipher with a 128-bit block and 128-, 192- or 256-bit keys (10, 12 or 14 rounds). Each round applies **SubBytes** (a non-linear S-box built from inversion in $GF(2^8)$), **ShiftRows**, **MixColumns** (a linear mix over $GF(2^8)$) and **AddRoundKey** (XOR with a round key from the key expansion). The same key encrypts and decrypts, so AES is fast but needs a way to share the key. A block cipher needs a **mode of operation** for data longer than one block. In ECB every block is encrypted independently. CBC and CTR use an IV or a nonce so that equal blocks produce different ciphertext.

2.2.2 Elliptic Curve Cryptography (ECC)

ECC works with the points of a curve $y^2 = x^3 + ax + b$ over a prime field F_p. Points can be added (chord-and-tangent rule), and repeated addition gives scalar multiplication. A private key is a random integer n and the public key is the point $\mathbf{P = n \cdot G}$ for a fixed base point G. Computing n from P is the **elliptic-curve discrete logarithm problem (ECDLP)**, which has no known efficient solution. ECC therefore offers the same security as RSA with much smaller keys: NIST SP 800-57 equates 256-bit ECC with 3072-bit RSA (128-bit security), 384-bit ECC with 7680-bit RSA and 512-bit ECC with 15360-bit RSA.

2.3 Working methodology of the base scheme

The algorithm is given in two boxes of the paper (Section 4.3), and the architecture in Figures 3 and 5:

- **Box 1 – public key generation:** select a prime n, select a private number n(a) < n, compute a curve point G, and return the public key $\mathbf{P = n(a) \cdot G}$.
- **Box 2 – encryption/decryption:** take the input file; "add the key generated by ECC which is the public key"; perform AES encryption "on the input file by using the public key which is generated by ECC"; upload; to decrypt, the file is "translated by using the public key given by ECC".
- **Architecture (Fig. 3/5):** the user's request goes to the Cloud Service Provider (CSP), which performs "verification" and then encryption (or decryption) before storing to (or returning from) the cloud server.

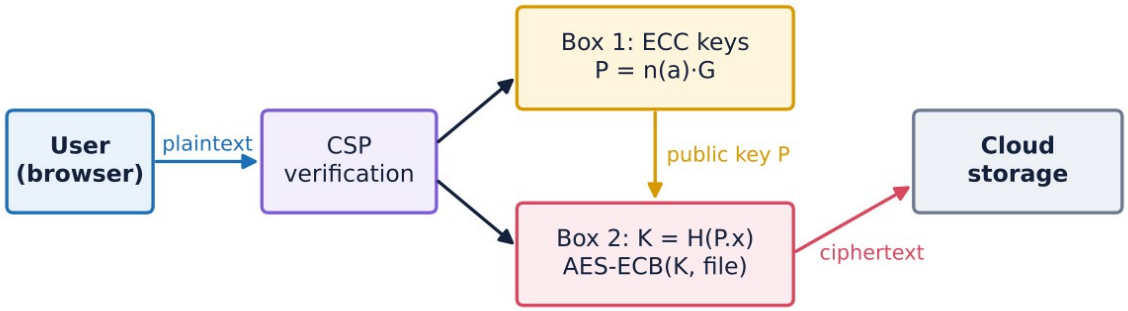


Figure 1. Base scheme as described in the paper (our rendering). The AES key is a function of the public key P only.

2.4 Results reported by the paper

The paper evaluates encryption and decryption time on text data with key sizes of 64, 128, 192 and 256 bits (Python, Intel i7-8550U). It reports that the hybrid model is the fastest, and also reports the avalanche effect, key sizes compared with RSA, and power consumption.

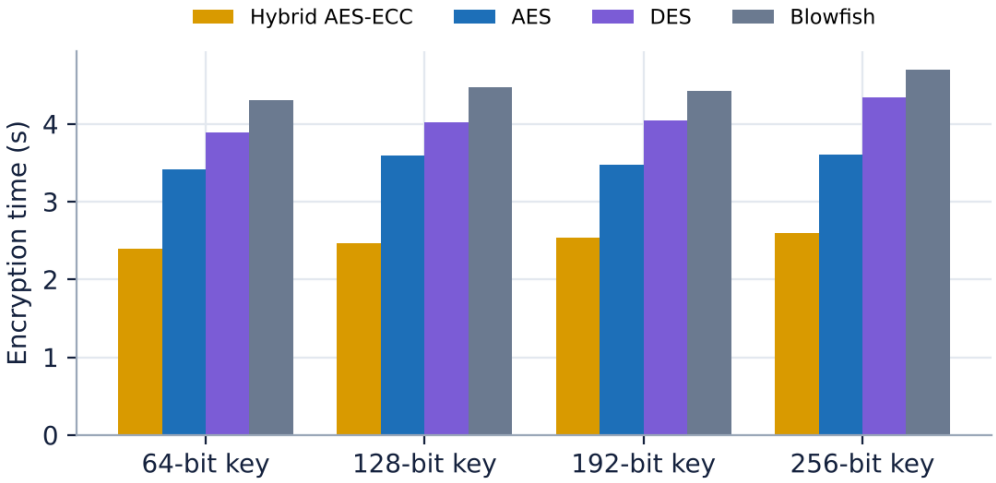


Figure 2. Encryption time reported in the paper (Table 7 of the paper), re-plotted.

Table 2. Main results reported by Rehman et al. (2021).

Metric (paper)	Hybrid AES-ECC	AES	DES	Blowfish
Encryption time, 64–256-bit keys (s)	2.40 – 2.60	3.42 – 3.60	3.89 – 4.34	4.31 – 4.70
Decryption time, 64–256-bit keys (s)	1.64 – 2.10	2.69 – 3.08	3.11 – 3.58	3.89 – 4.25
Avalanche, 1-bit key / plaintext change	0.61 / 0.64	0.51 / 0.55	–	0.29 / 0.18
Key size claim (Table 5)	ECC 256 ≈ RSA 3024 (1:12)			

2.5 Advantages and applications

- Simple design that is easy to implement: one ECC key pair per user and one AES encryption per file.
- ECC gives small keys and fast key generation, which suits mobile and IoT clients.
- Applications suggested by the paper: cloud storage for personal data, organisational documents and data accessed from weakly secured devices.

3. Implementation of the Existing Approach

3.1 Tools, technologies and languages

Table 3. Tools and technologies.

Layer	Technology	Purpose
Crypto (base scheme)	Node.js v22.23.4 `crypto` (OpenSSL 3.5.9)	P-256 key generation, SHA-256, AES-128-ECB/CBC
Crypto (proposed)	W3C Web Crypto API (browser and Node.js)	AES-GCM, ECDH, ECDSA, HKDF, PBKDF2, SHA-256
Server	Express 4, Mongoose 8, Multer, JWT, bcrypt	REST API, file upload, authentication, Attack Lab, live log (SSE)
Database	MongoDB 7 with GridFS	users, file metadata/manifests, ciphertext blobs
Client	React 18, React Router, Recharts, Vite 7	UI, client-side encryption, Crypto Inspector, charts
Testing / analysis	node:test, Playwright, Python (matplotlib)	unit tests, end-to-end UI test + screenshots, figures

3.2 Implementation methodology

The repository linked in the paper (github.com/sabarehmanciit/Hybrid-AES-ECC-Model) contains only a README, so we implemented the boxes ourselves. Where the paper is silent, we chose the most charitable concrete option, so that every weakness we report is in the design and not in our choices:

Table 4. Implementation decisions for details the paper leaves open.

Not specified in the paper	Our choice	Reason
Elliptic curve	NIST P-256 (prime256v1)	standard 128-bit-security curve
How the AES key is obtained from the ECC public key	K = SHA-256(x-coordinate of P), truncated to 128 bits	a proper hash. Any deterministic function of P has the same flaw
AES mode, IV, padding	AES-128-ECB with PKCS#7 (variant: CBC with fixed IV)	Box 2 mentions no IV or nonce and nothing is stored with the file
Where encryption happens	On the server (CSP) after login verification	as in the paper's Figures 3 and 5

```
// crypto-core/src/base_aes_ecc.js  (excerpt)
export function baseKeygen() {                          // Box 1: P = n(a)·G
  const ecdh = crypto.createECDH('prime256v1'); ecdh.generateKeys();
  return { priv: ecdh.getPrivateKey(), pub: ecdh.getPublicKey(null, 'uncompressed') };
}
export function deriveAesKeyFromPublic(pub, bits = 128) {   // Box 2: key from PUBLIC
key
  return sha256(pub.subarray(1, 33)).subarray(0, bits / 8);
}
export function baseEncrypt(plain, pub) {
  const c = crypto.createCipheriv('aes-128-ecb', deriveAesKeyFromPublic(pub), null);
  return Buffer.concat([c.update(plain), c.final()]);
}
```

3.3 Experimental setup

- Machine: Intel(R) Xeon(R) Processor @ 2.10GHz (4 cores), Linux 6.18.44-fc-v185; Node.js v22.23.4 with OpenSSL 3.5.9.
- Data: synthetic employee/payment records (1–65 KiB, text with repeated structure), random binary files, a 512×384 flat-colour image (BMP, 589,824 pixel bytes), and a 96-byte payment order.
- Performance: file sizes from 1 KiB to 32 MiB; median of 7–25 repetitions after a warm-up run, with garbage collection forced between runs.
- All experiments are in `app/bench/run.js` and use exactly the code that the web application uses. Results are written to `results.json`, which the slides and this report read.

3.4 Results of the implementation

The base scheme works end-to-end in our application: all 100 test files encrypt and decrypt correctly. Our implementation confirms the paper's efficiency argument for ECC: generating a P-256 key pair takes 0.28 ms, against 145 ms for RSA-3072 at the same security level, and AES-128 encrypts a 16 MiB file in 17.2 ms (≈928 MB/s). We cannot reproduce the paper's absolute timings in seconds, because its data set and code are not available.

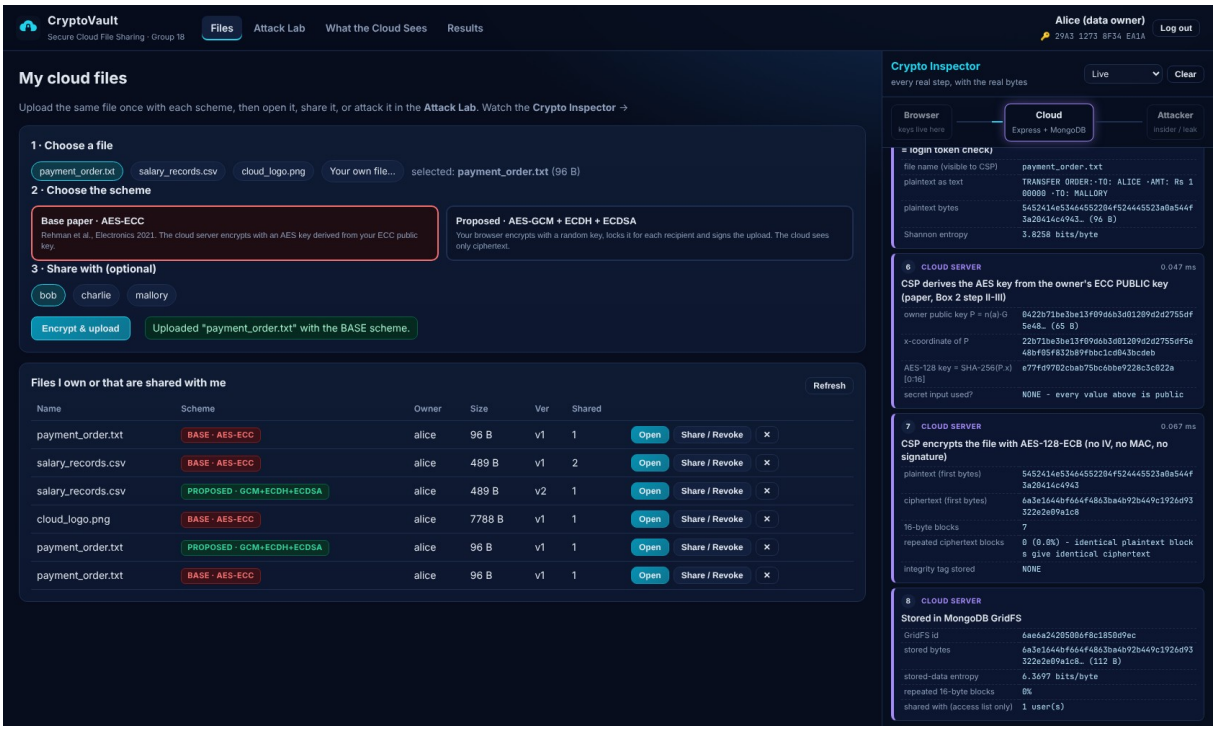


Figure 3. Uploading a file with the base scheme in CryptoVault. The Crypto Inspector (right) shows the CSP receiving the plaintext and deriving the AES key from the public key.

4. Challenges and Limitations

We identified three challenges. Each one was demonstrated with a working attack in the Attack Lab of our application and measured in the experiment harness.

4.1 Challenge C1 – The AES key is derived from the public key

What is wrong. Box 2 uses the ECC public key as the source of the AES key ("AES encryption is performed on the input file by using the public key which is generated by ECC"). A public key is published by design; the paper itself states that "the public key is known by every person". Anyone who obtains the ciphertext can therefore recompute the AES key and decrypt. The ECC private key n(a) is never used, so the hardness of the ECDLP gives no protection.

Why it is important (course link). Kerckhoffs' principle says that a system must stay secure when everything except the key is public. Here the key itself is public. In Shannon's terms, the **key equivocation** $H(K \mid P)$ is 0 bits: after seeing the public key, the attacker has no uncertainty about the AES key. No choice of cipher (AES-128 or AES-256) can compensate for a key with zero entropy.

Evidence. Attack 1 gives an attacker only the stored ciphertext and the owner's public key from the directory. It recovered **100 of 100 files** (Section 7.2).

4.2 Challenge C2 – No integrity or authentication; ECB-style encryption

What is wrong. The abstract claims the scheme ensures "authentication and data integrity", but the algorithm contains no MAC, no hash, no signature and no authenticated mode. AES is used without an IV, so the same plaintext block always gives the same ciphertext block, which is ECB behaviour. Three consequences follow:

- **Malleability:** the cloud can flip bits, delete blocks, swap blocks or splice blocks from another file, and decryption still succeeds. In the payment-order demo, swapping two ciphertext blocks swaps the amounts of Alice and Mallory.

- **Pattern leakage:** equal plaintext blocks give equal ciphertext blocks, so the shapes in an image stay visible after encryption.
- **Deterministic encryption:** uploading the same file twice gives identical ciphertexts, which tells an observer that two files are equal.

Evidence. 0 of 1,250 tampering attempts were detected, and 1,167 produced silently corrupted output. 98.2% of the image's ciphertext blocks are repeats (Section 7.3–7.4).

4.3 Challenge C3 – No real sharing, revocation or fair evaluation

What is wrong. Encryption is done by the CSP (Figures 3 and 5), so the provider receives the plaintext and can compute the key. This contradicts the stated goal of working "without involving a third party". Sharing can only be an access-control list on the server: there are no per-user keys, so removing a user changes no key and **revocation is impossible** cryptographically. A user who was once allowed, or anyone else, can still decrypt any copy with the public key.

Evaluation issues. The paper's evaluation also has methodological problems that make its comparisons unreliable:

- AES is reported with a "64-bit key", but FIPS-197 defines AES only for 128-, 192- and 256-bit keys.
- Table 5 lists RSA equivalents of 3024, 7068 and 16360 bits; NIST SP 800-57 gives 3072, 7680 and 15360 bits.
- An avalanche effect above 50% (61–64%) is read as "more secure". The strict avalanche criterion (Webster & Tavares) asks for 50%; any deviation in either direction indicates bias.
- The "power consumption (W)" values in Tables 10 and 11 are identical to the encryption and decryption times in Tables 7 and 8, which suggests power was not measured separately.
- The hybrid model (AES plus ECC operations) is reported to be faster than AES alone, which is only possible if different implementations were compared.
- The linked code repository contains only a README, so the results cannot be reproduced.

Table 5. Summary of the challenges.

#	Challenge	Affects	Impact
C1	AES key derived from public ECC key	Confidentiality	Any outsider or insider decrypts every file. Security reduces to zero.
C2	No MAC/signature; ECB-style mode	Integrity, authentication, confidentiality	Silent tampering (financial/medical data), pattern and equality leakage
C3	CSP encrypts; no per-user keys; flawed evaluation	Key management, usability, scalability	No revocation, provider must be fully trusted, misleading performance claims

5. Proposed Approach: Authenticated Hybrid Envelope (AHE)

5.1 Selected challenges

The guidelines ask for at least one challenge to be solved. AHE solves **C1** and **C2** completely and solves the cryptographic part of **C3** (end-to-end encryption, per-user sharing, revocation). The evaluation issues of C3 are addressed by our own measurement methodology (Section 6.3).

5.2 Design principles

1. **Never derive a secret from public data.** Every file gets a fresh random 256-bit data-encryption key (DEK).
2. **Encrypt and authenticate together.** Use an AEAD mode (AES-GCM) so that any change to the ciphertext is detected (Bellare & Namprempre).
3. **Encrypt the key separately for each recipient** with Diffie-Hellman on the curve (ECIES-style key encapsulation).
4. **Sign what you upload.** The owner signs a manifest that binds the ciphertext hash, the recipients' wrapped keys and the version.
5. **Encrypt at the edge.** All cryptography runs in the users' browsers; the server stores only opaque bytes (zero-knowledge server).

5.3 Building blocks

- **DEK:** 32 bytes from the browser's CSPRNG (`crypto.getRandomValues`). The key has 256 bits of entropy and is never stored in plaintext.
- **AES-256-GCM:** the file is split into 1 MiB chunks. Nonce = 64-bit random prefix ‖ 32-bit chunk counter. The associated data binds `scheme | fileId | version | chunk index | last-chunk flag`, which prevents reordering, truncation and splicing between files or versions. GCM = CTR mode for secrecy + GHASH (multiplication in GF(2^{128})) for the 128-bit tag.
- **Key wrapping (ECDH + HKDF):** for recipient j with public key Q_j the sender picks an ephemeral r, sends R = r·G, computes S = r·Q_j, and derives KEK = HKDF-SHA256(S, salt = fileId, info = context ‖ R). The DEK is encrypted with AES-GCM under KEK. The recipient computes the same S = d_j·R with its private key d_j. This is the Diffie-Hellman exchange from Unit 6, performed on elliptic curves.
- **Signature:** ECDSA over P-256 with SHA-256 on the canonical JSON manifest {fileId, version, size, chunks, nonce prefix, encrypted name, SHA-256(ciphertext), recipients' wrapped keys}.
- **Key vault:** each user's ECDH and ECDSA private keys are encrypted in the browser with AES-GCM under PBKDF2-SHA256(password, random salt, 600,000 iterations). The login token is a different PBKDF2 output, and the server stores only bcrypt(token), so it never learns the password and cannot open the vault.

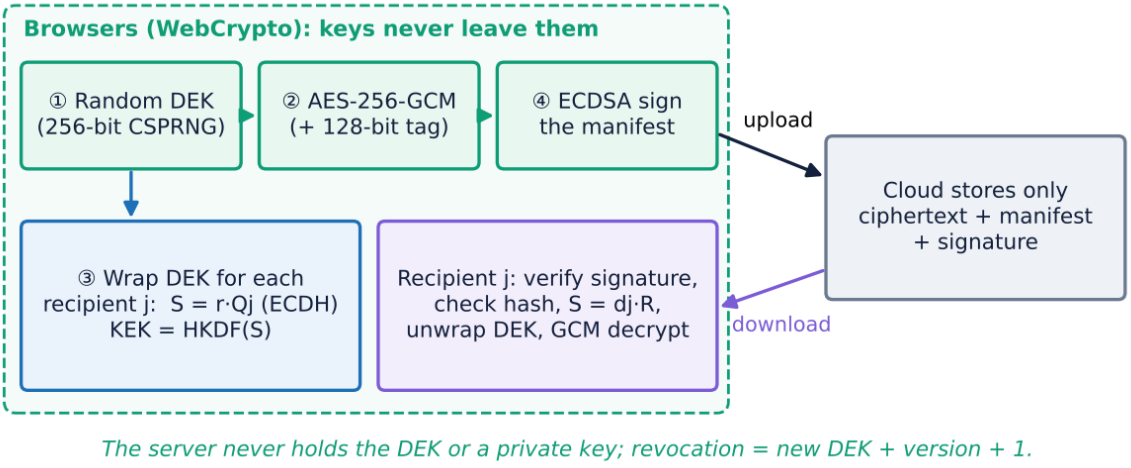


Figure 4. Proposed AHE scheme: upload (top row), per-recipient key wrapping and recipient-side verification/decryption.

5.4 Algorithms (pseudocode)

```
Algorithm 1  UPLOAD(file, recipients, owner)
  DEK ← CSPRNG(256 bit);  prefix ← CSPRNG(64 bit)
  for i, chunk in split(file, 1 MiB):
      C_i ← AES-GCM_DEK(nonce = prefix‖i, AAD = fileId‖version‖i‖last, chunk)
  for each recipient j (incl. owner):
      r ← random; R ← r·G; S ← r·Q_j; KEK ← HKDF(S, fileId, ctx‖R)
      wrap_j ← AES-GCM_KEK(DEK)
  M ← {fileId, version, size, prefix, Enc(name), SHA-256(C), [wrap_j, R_j]}
  σ ← ECDSA_sign(sk_owner, M);  erase DEK;  send (C, M, σ) to cloud

Algorithm 2  OPEN(C, M, σ, user j)
  if not ECDSA_verify(pk_owner, M, σ)        → BLOCK (manifest changed)
  if SHA-256(C) ≠ M.ctHash                   → BLOCK (ciphertext changed)
  if no wrap for j                           → DENY  (not a recipient / revoked)
  S ← d_j·R_j;  DEK ← AES-GCM-Dec_KEK(wrap_j)  (tag check)
  for each chunk: P_i ← AES-GCM-Dec_DEK(C_i)  (tag check, else BLOCK)

Algorithm 3  SHARE(M, new user k):  DEK ← unwrap(own entry); add wrap_k; re-sign M
Algorithm 4  REVOKE(user k):  P ← OPEN(...); UPLOAD(P, recipients \ {k}) as version+1
```

5.5 Security analysis (informal)

- **C1 fixed.** The attacker sees C, M, σ and every public key. Obtaining the DEK requires either S = r·Q_j = d_j·R, which is the elliptic-curve Diffie-Hellman problem, or a brute-force search over 2^{256} keys. The DEK is independent of all public values, so H(DEK | public data) = 256 bits instead of 0.
- **C2 fixed.** Changing any ciphertext byte changes SHA-256(C) and the GCM tag of that chunk. Changing the manifest invalidates the ECDSA signature, and creating a new valid signature requires the owner's private key (EUF-CMA security of ECDSA). A random fresh nonce and DEK per upload make encryption non-deterministic, so equal files and equal blocks give unrelated ciphertexts.
- **C3 fixed (cryptographic part).** The server never receives plaintext, a DEK or a private key. A new user receives one extra wrapped key, without re-encryption. Revocation re-encrypts under a new DEK, and because the version is bound into the associated data, a revoked user's old wrapped key cannot be re-used on the new version.

5.6 Limitations of the proposed approach

- **Public-key substitution:** a malicious server could hand out a fake public key. We display SHA-256 key fingerprints for out-of-band comparison; a key-transparency log would be the stronger fix.
- **Metadata:** file sizes, upload times, access patterns and the number of recipients remain visible to the server.
- **Already-downloaded data:** revocation protects new versions only; a user can keep a copy of what they already decrypted (true for any system).
- **Rollback:** the server could serve an older, validly signed version; clients would need to remember the highest version they have seen.
- **No post-quantum security:** ECDH and ECDSA can be broken by a large quantum computer; ML-KEM (FIPS 203) would replace ECDH in a future version.

6. Implementation of the Proposed Approach

6.1 Implementation process and code structure

The cryptography lives in one shared module, `crypto-core`. The **same file** runs in the React client (browser WebCrypto) and in Node.js (seed script, tests, benchmarks), so the benchmarked code is the

demonstrated code. Every function accepts an optional `trace` callback. The client's **Crypto Inspector** panel shows each step with its real values (keys, nonces, shared secrets, tags, hashes, signatures) and can play them back one at a time for the demonstration.

Table 6. Main modules of the implementation.

Path	Content
`app/crypto-core/src/base_aes_ecc.js`	Base scheme (Box 1 + Box 2), traced versions for the CSP
`app/crypto-core/src/proposed_ahe.js`	AHE: key generation, vault, wrap/unwrap, encrypt/decrypt, share, re-key, attack helper
`app/crypto-core/src/analysis.js`	Shannon entropy, repeated-block ratio, chi-square, bit difference (avalanche)
`app/server/src/routes/*.js`	auth (PBKDF2 token + bcrypt + JWT), files (base & proposed), lab (attacks), SSE live log
`app/client/src/pages/*.jsx`	Files, Attack Lab, What the Cloud Sees, Results; `components/Inspector.jsx`
`app/bench/run.js`	Experiments E1–E9 (security + performance)
`app/crypto-core/test/crypto.test.js`	Unit tests: round trip, tamper detection, attacks, sharing, revocation, vault

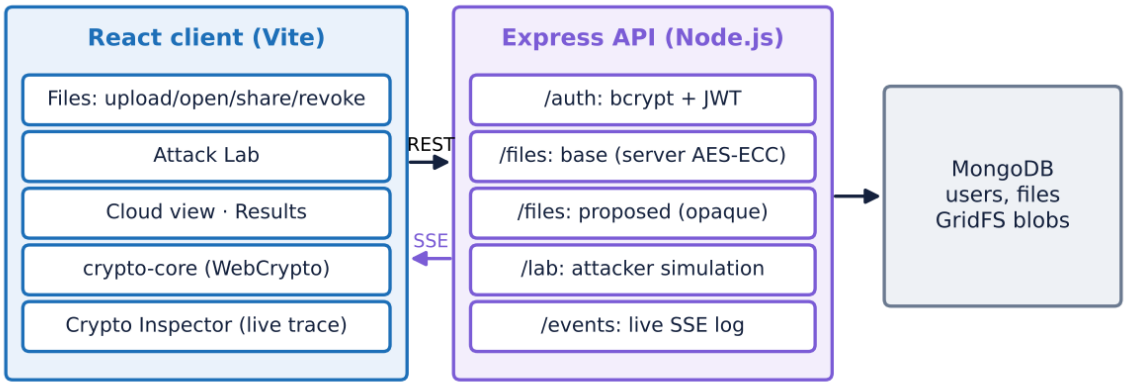


Figure 5. Architecture of the CryptoVault demonstration application.

6.2 Application walk-through

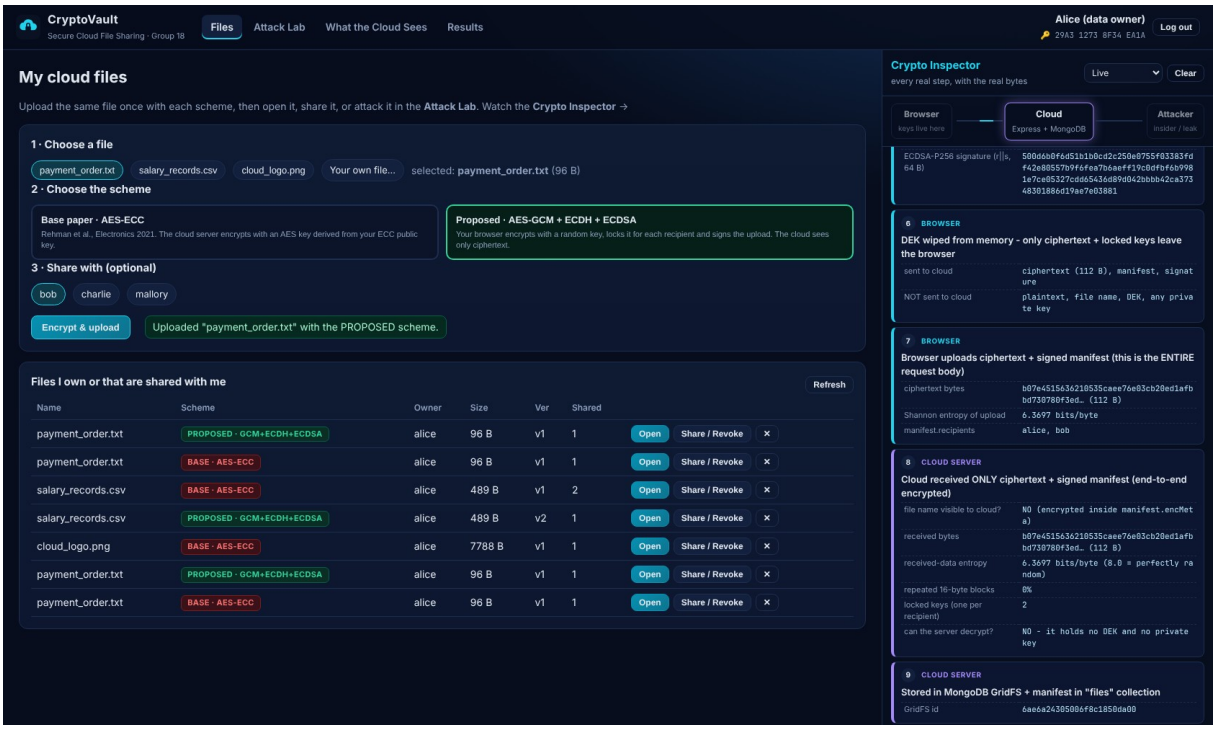


Figure 6. Upload with the proposed scheme. The Inspector shows the browser generating the DEK, encrypting with AES-GCM, locking the key for each recipient with ECDH, and signing; the server reports that it received only high-entropy ciphertext.

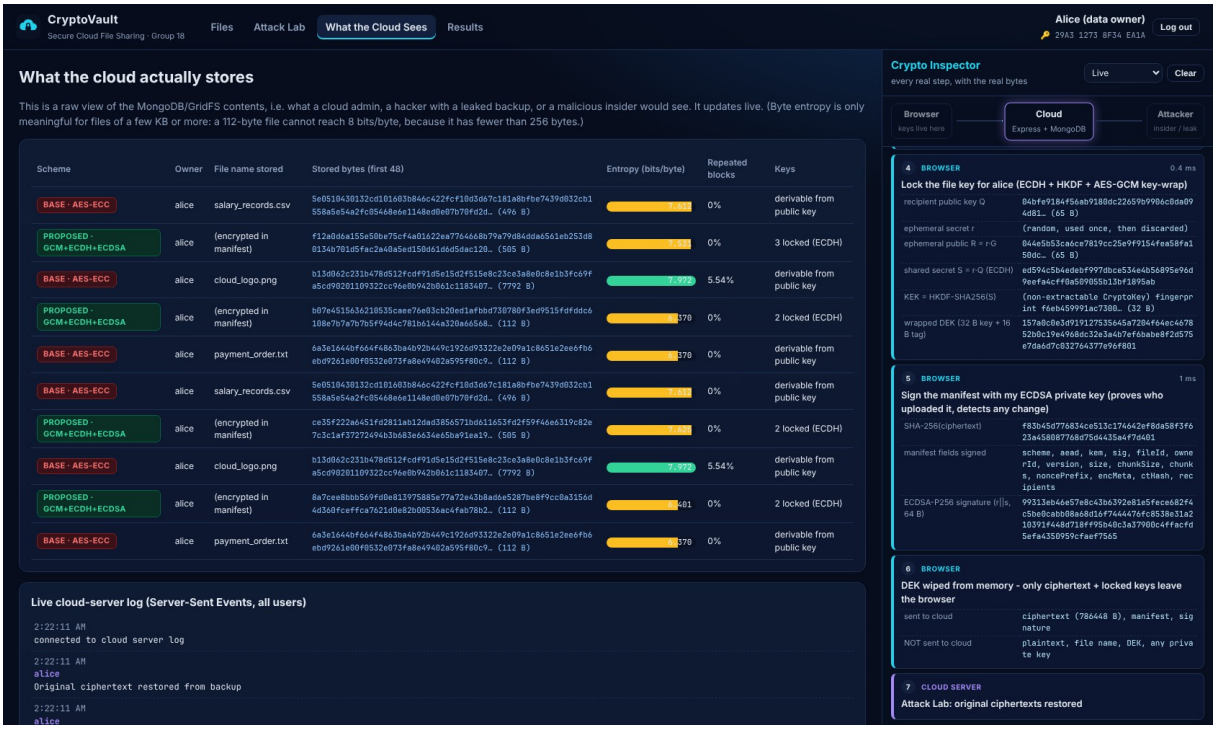


Figure 7. "What the Cloud Sees": the raw MongoDB/GridFS contents. Base files expose their names and have keys derivable from public keys; proposed files expose neither.

6.3 Experimental setup (security and performance)

Table 7. Experiments (all in app/bench/run.js).

Exp.	What is measured	Trials / sizes
E1	Decryption by an attacker who knows only public data	100 files (1–65 KiB)
E2	Detection of 5 tampering types (bit flip, block swap, block delete, splice, file swap / manifest edit)	250 per type, 1,250 total
E3	Pattern leakage on an image: entropy, repeated blocks, chi-square	512×384 image
E4	Determinism: same file encrypted twice	4 KiB document
E5	Avalanche: 1-bit key / plaintext change (block and whole file)	300 trials
E6	Encryption/decryption time and throughput	1 KiB – 32 MiB
E7	Sharing cost vs number of recipients; adding a recipient later	1–100 recipients, 1 MiB
E8	Revocation (re-key) cost	64 KiB – 16 MiB
E9	Cost of primitives (P-256 vs RSA-3072)	50–100 runs

7. Results and Analysis

7.1 Summary

Table 8. Base vs. proposed scheme under identical experiments.

Test	Base AES-ECC	Proposed AHE
Files decrypted using only public data (E1)	100 / 100	0 / 100
Tampering detected (E2, 1,250 attacks)	0.0 %	100 %
Repeated 16-byte blocks in encrypted image (E3)	98.15 %	0 %
Ciphertext entropy, bits/byte (E3)	6.553	7.9997
Chi-square vs uniform, 255 d.o.f. (E3)	1,512,580	242
Same file twice: bits that differ (E4)	0 % (identical)	49.84 %
1-bit plaintext change, whole 4 KiB file (E5)	0.19 %	50.01 %
Revocation of a user (E8)	impossible	7.6 ms per MiB
Encrypt 1 MiB / 16 MiB (E6)	0.99 / 17.2 ms	4.97 / 41.0 ms

7.2 Attack 1: decryption with public information (C1)

With only the stored ciphertext and the owner's public key, the attacker decrypted **all 100 base files**. Against the proposed scheme the same attacker, given the ciphertext, the full manifest and every public key in the directory, tried 1,000 candidate keys derived from all public values, and **none** passed the GCM tag check.

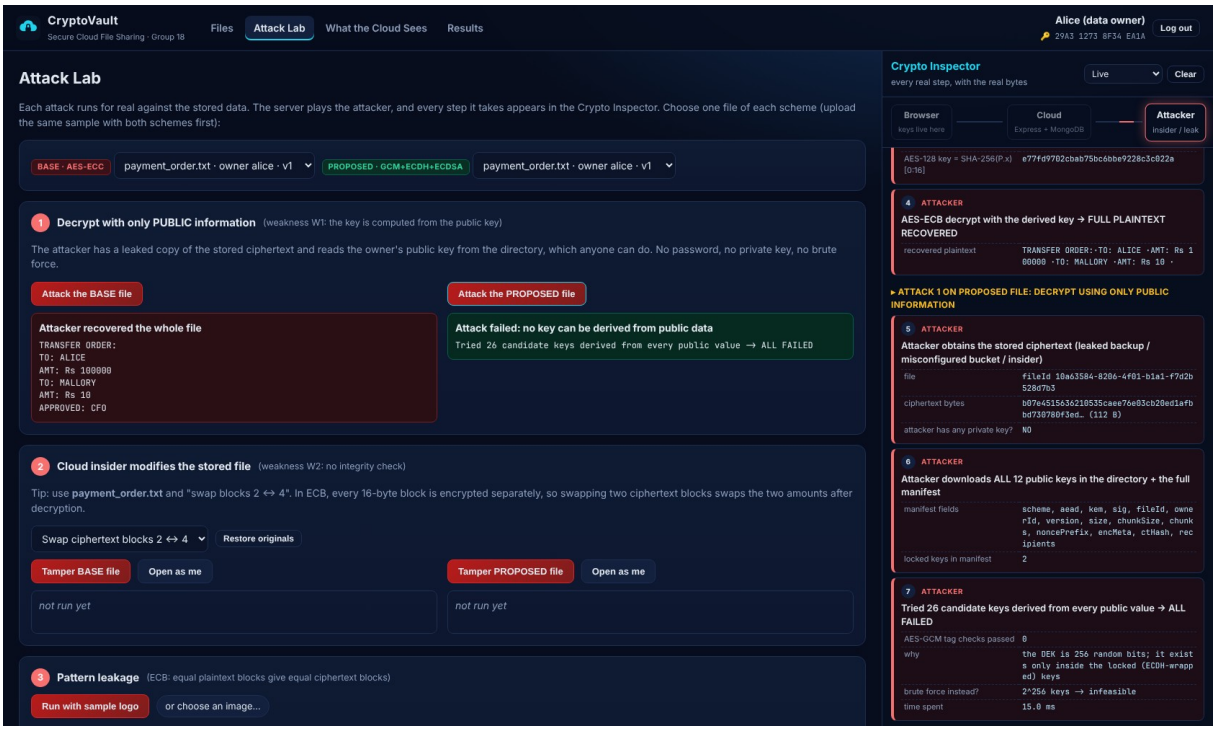


Figure 8. Attack Lab, attack 1: the base file is fully recovered from public data, while every key candidate fails against the proposed file. The steps appear in the Inspector.

7.3 Attack 2: tampering by the cloud (C2)

The base scheme raised no error for any of the 1,250 modifications. In 1,167 cases it returned corrupted plaintext as if it were valid. In the remaining cases the modification happened to leave the output unchanged, e.g. two identical ECB blocks were swapped, which is itself a symptom of ECB. The proposed scheme detected **every** modification, through the ECDSA signature, the SHA-256 ciphertext hash or the GCM tag, and released no data.

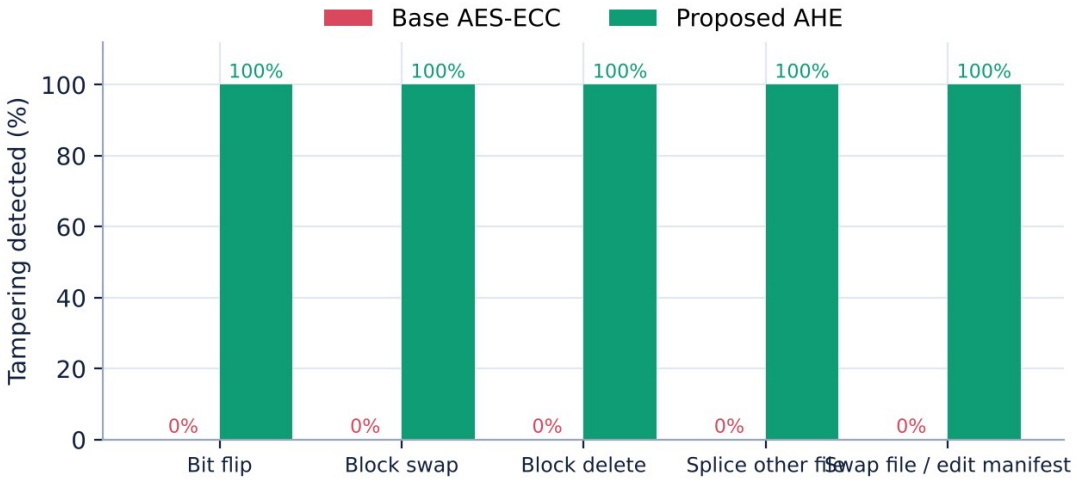


Figure 9. Tampering detection rate per attack type.

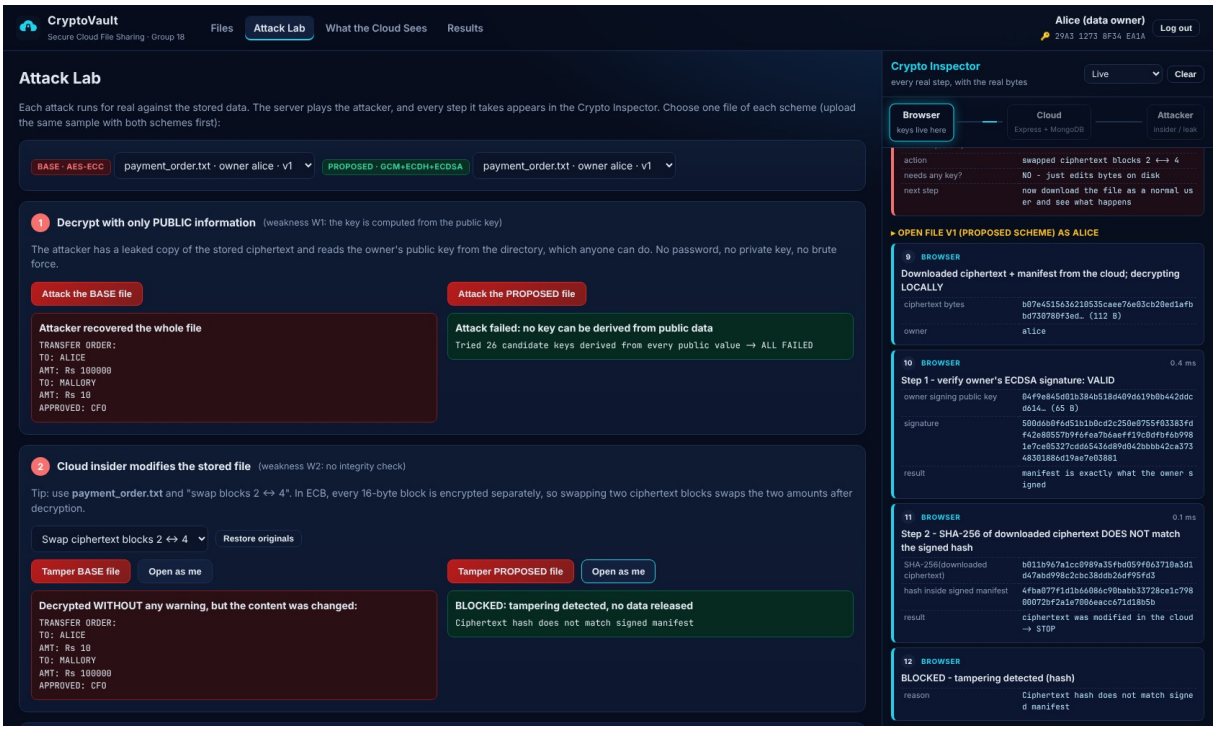


Figure 10. Payment-order demo: swapping ciphertext blocks 2 and 4 swaps the amounts of Alice and Mallory under the base scheme without any warning; the proposed scheme blocks the file.

7.4 Pattern leakage, entropy and determinism

Encrypting the flat-colour image with the base scheme leaves the picture recognisable (Figure below). Its ciphertext entropy is only 6.55 bits/byte, and 98.2% of its blocks are repeats, exactly as many as in the plaintext. The proposed ciphertext reaches 7.9997 bits/byte (the maximum is 8) with no repeated block, and its chi-square statistic (242) is in the range expected for uniformly random bytes (≈255). Uploading the same document twice gives byte-identical ciphertexts in the base scheme, and ciphertexts that differ in 49.8% of their bits in the proposed scheme.

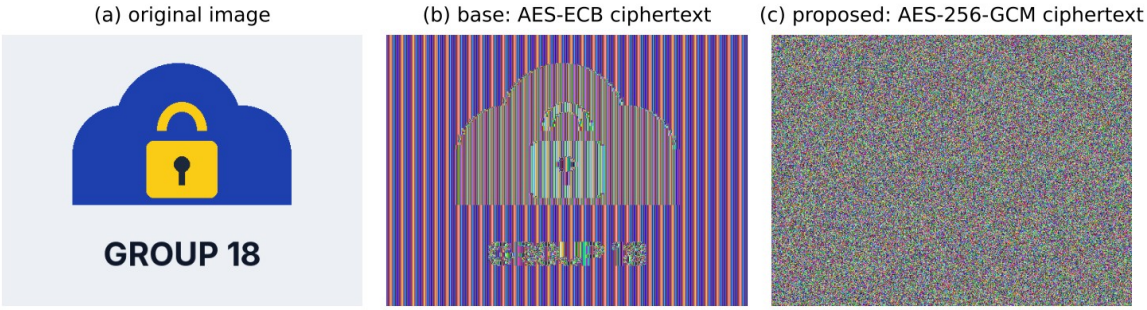


Figure 11. The same image encrypted by both schemes.

7.5 Avalanche effect re-evaluated

Measured correctly, AES changes 49.85% of ciphertext bits for a 1-bit key change and 49.86% for a 1-bit plaintext change: essentially the ideal 50%. The paper's figures of 61–64% for its hybrid model are therefore not “more secure”; they indicate a measurement problem. At the level of a whole file, a 1-bit change alters only 0.19% of the base ciphertext (one ECB block), compared with 50.01% for the proposed scheme, which re-encrypts with a fresh key and nonce.

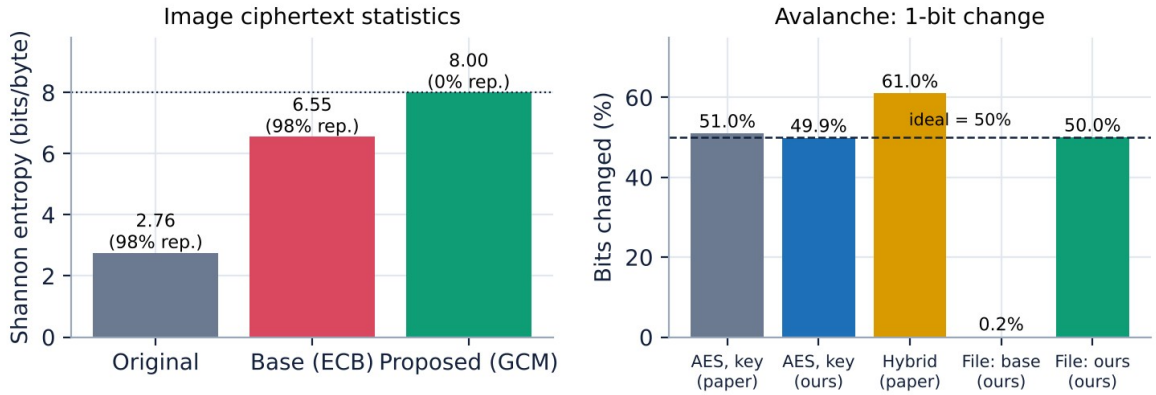


Figure 12. Left: entropy and repeated blocks of the encrypted image. Right: avalanche measurements (dashed line = ideal 50%).

7.6 Performance

The proposed scheme has a fixed cost of about 2.9 ms per file for encryption and 2.0 ms for decryption, made up of two ECDH key wraps (owner + one recipient), one ECDSA signature and the key imports (one P-256 operation costs ≈0.27 ms). Per byte, it adds the GCM tag computation and a SHA-256 over the ciphertext. For a 16 MiB file encryption takes 41.0 ms instead of 17.2 ms, i.e. 390 MB/s instead of 928 MB/s. This is still far faster than typical network upload speeds, so users would not notice the difference. ECC is also the right choice for key wrapping: P-256 key generation takes 0.28 ms against 145 ms for RSA-3072.

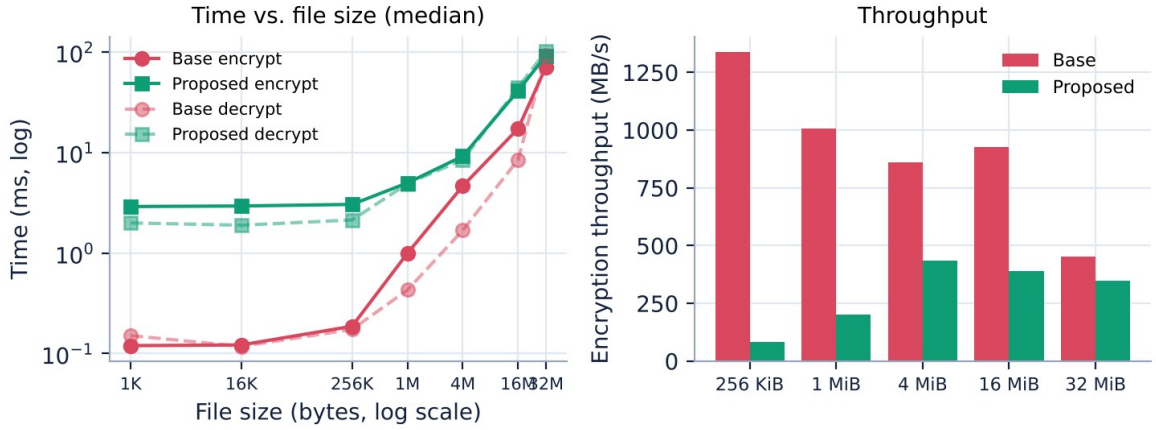


Figure 13. Encryption/decryption time vs file size (log-log) and encryption throughput.

Table 9. Median timings (E6).

Size	Base enc (ms)	Base dec (ms)	AHE enc (ms)	AHE dec (ms)	AHE enc (MB/s)
1 KiB	0.12	0.15	2.90	1.99	0.3
16 KiB	0.12	0.12	2.94	1.89	5.3
256 KiB	0.19	0.17	3.04	2.13	82.2
1 MiB	0.99	0.43	4.97	4.88	201.0
4 MiB	4.64	1.68	9.18	8.44	435.9
16 MiB	17.25	8.46	41.01	44.34	390.2
32 MiB	70.42	92.23	91.61	101.26	349.3

7.7 Sharing and revocation

Sharing cost grows linearly with the number of recipients (one ECDH wrap and 216 bytes of manifest each): encrypting and sharing a 1 MiB file with 100 recipients takes 78 ms. Adding a recipient later needs no re-encryption and takes 2.7 ms. Revocation re-encrypts the file under a new DEK: 7.6 ms for 1 MiB and 72.5 ms for 16 MiB. In the application, the revoked user can still download the new ciphertext (we allow this on purpose) and is still denied, because there is no wrapped key for them in version 2. Revocation is therefore enforced by cryptography and not by the server.

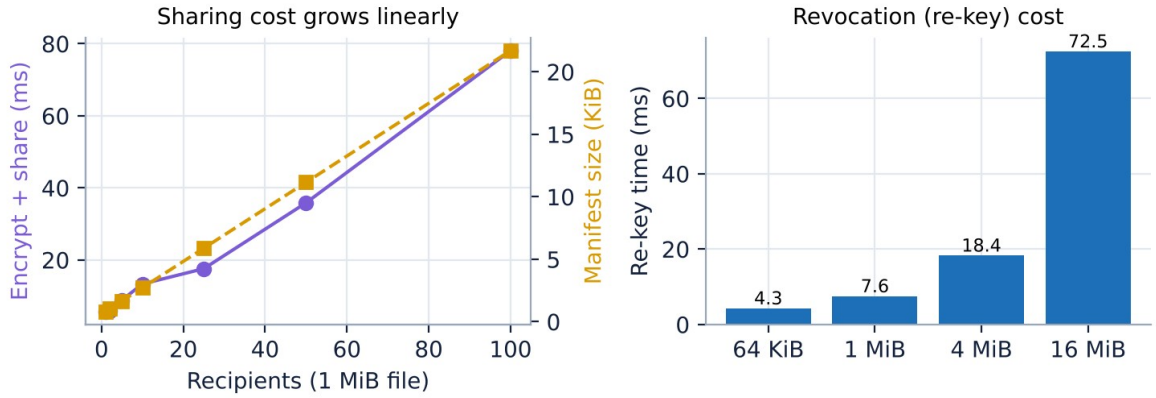


Figure 14. Sharing cost vs number of recipients, and re-key (revocation) cost vs file size.

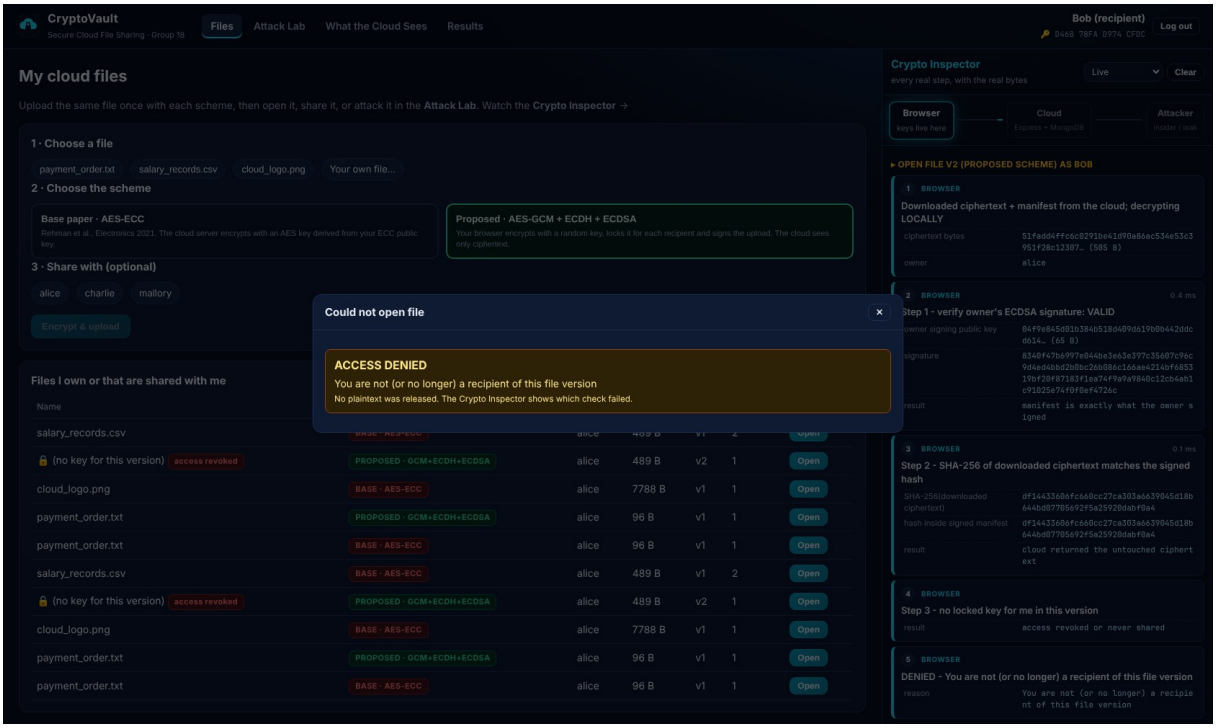


Figure 15. After revocation, Bob's browser receives version 2 but finds no wrapped key for him: access is denied by cryptography.

7.8 Does the proposed approach address the challenges?

Table 10. Challenge-by-challenge outcome.

Challenge	Before (base)	After (AHE)	Solved?
C1 key from public data	100/100 files recovered	0/100; DEK has 256-bit entropy	Yes
C2 integrity / ECB	0% tampering detected; 98% blocks repeat	100% detected; 0% repeat; signed uploads	Yes

Challenge	Before (base)	After (AHE)	Solved?
C3 sharing / revocation	CSP sees plaintext; no revocation	zero-knowledge server; per-user wrap; re-key	Yes (crypto part)
C3 evaluation validity	unreproducible, inconsistent tables	open harness, medians, all numbers regenerated	Addressed

8. Conclusion

We re-implemented the Hybrid AES-ECC model of Rehman et al. (2021) from its published algorithm and evaluated it with real attacks instead of timing alone. The scheme is fast, but its central design decision, deriving the AES key from the ECC public key, removes all confidentiality: anyone who can read the public key can decrypt every file. It also provides none of the integrity or authentication it claims, leaks data patterns, gives the provider the plaintext, and cannot revoke users. Several of its reported results are not methodologically sound.

Our Authenticated Hybrid Envelope keeps the hybrid AES + ECC idea but uses each primitive correctly: a random per-file key, AES-256-GCM for authenticated encryption, ECDH-based key wrapping for each recipient, ECDSA signatures and client-side encryption. Under identical experiments every attack failed, every tampering attempt was detected, and the ciphertext became statistically indistinguishable from random, at a cost of a few milliseconds per file. All components come from the course syllabus. The project shows that the security of a hybrid scheme depends on how the primitives are combined, not only on which primitives are used.

Future scope: post-quantum key encapsulation (ML-KEM, FIPS 203) in a hybrid with ECDH; a key-transparency log to stop public-key substitution; client-side version tracking against rollback; and proxy re-encryption or attribute-based encryption to make revocation in very large groups cheaper.

9. References

[1] S. Rehman, N. T. Bajwa, M. A. Shah, A. O. Aseeri, A. Anjum, “Hybrid AES-ECC Model for the Security of Data over Cloud Storage,” Electronics, vol. 10, no. 21, 2673, 2021. doi:10.3390/electronics10212673. (Base paper)

[2] NIST, FIPS PUB 197, “Advanced Encryption Standard (AES),” 2001 (updated 2023).

[3] M. Dworkin, NIST SP 800-38A, “Recommendation for Block Cipher Modes of Operation: Methods and Techniques,” 2001.

[4] M. Dworkin, NIST SP 800-38D, “Recommendation for Block Cipher Modes of Operation: Galois/Counter Mode (GCM) and GMAC,” 2007.

[5] NIST, FIPS PUB 186-5, “Digital Signature Standard (DSS),” 2023.

[6] NIST SP 800-56A Rev. 3, “Recommendation for Pair-Wise Key-Establishment Schemes Using Discrete Logarithm Cryptography,” 2018.

[7] NIST SP 800-57 Part 1 Rev. 5, “Recommendation for Key Management: Part 1 – General,” 2020.

[8] NIST, FIPS PUB 180-4, “Secure Hash Standard (SHS),” 2015.

[9] H. Krawczyk, P. Eronen, RFC 5869, “HMAC-based Extract-and-Expand Key Derivation Function (HKDF),” IETF, 2010.

[10] K. Moriarty, B. Kaliski, A. Rusch, RFC 8018, “PKCS #5: Password-Based Cryptography Specification Version 2.1,” IETF, 2017.

[11] V. Shoup, “A Proposal for an ISO Standard for Public Key Encryption (version 2.1),” IACR Cryptology ePrint Archive, Report 2001/112, 2001.

[12] M. Bellare, C. Namprempre, “Authenticated Encryption: Relations among Notions and Analysis of the Generic Composition Paradigm,” ASIACRYPT 2000, LNCS 1976, pp. 531–545.

[13] C. E. Shannon, “Communication Theory of Secrecy Systems,” Bell System Technical Journal, vol. 28, no. 4, pp. 656–715, 1949.

[14] A. Kerckhoffs, “La cryptographie militaire,” Journal des sciences militaires, vol. IX, pp. 5–38, 161–191, 1883.

[15] A. F. Webster, S. E. Tavares, “On the Design of S-Boxes,” CRYPTO '85, LNCS 218, pp. 523–534, 1986.

[16] S. Vaudenay, “Security Flaws Induced by CBC Padding – Applications to SSL, IPSEC, WTLS…,” EUROCRYPT 2002, LNCS 2332, pp. 534–545.

[17] S. Kamara, K. Lauter, “Cryptographic Cloud Storage,” Financial Cryptography Workshops 2010, LNCS 6054, pp. 136–149.

[18] W3C, “Web Cryptography API,” W3C Recommendation, 26 January 2017.

[19] NIST, FIPS PUB 203, “Module-Lattice-Based Key-Encapsulation Mechanism Standard (ML-KEM),” 2024.

[20] OWASP, “Password Storage Cheat Sheet” (PBKDF2-HMAC-SHA256, 600,000 iterations), OWASP Cheat Sheet Series, accessed 2026.

[21] D. R. Stinson, Cryptography: Theory and Practice, 3rd ed., Chapman & Hall/CRC, 2006. (Course text book)

[22] B. A. Forouzan, D. Mukhopadhyay, Cryptography and Network Security, McGraw-Hill Education, 2011. (Course text book)

[23] W. Stallings, Cryptography and Network Security: Principles and Practice, 6th ed., Pearson, 2014.

[24] MongoDB Inc., “GridFS” documentation; Node.js Foundation, “Crypto” module documentation (Node.js 22), accessed 2026.

Appendix A. Running the demonstration

The application needs Node.js 20.19+ and a MongoDB database (MongoDB Atlas or a local server). The connection string is read from `app/server/.env`.

```
# 1. app/server/.env  (only MONGO_URI is required)
MONGO_URI=mongodb+srv://<user>:<password>@<cluster>.mongodb.net/cryptovault

# 2. install, test, run
cd app
npm install                 # installs server, client, crypto-core (npm workspaces)
npm test                    # 5 unit tests of the crypto core
npm run bench               # regenerates bench/results/results.json (~20 s)
npm start                   # builds the React client and serves everything on
http://localhost:5000
#   demo users are created on first start: alice / Alice@Cloud2026, bob / Bob@Cloud2026,
...
```

Appendix B. Division of work

Table 11. Work division (details in the separate task-division document).

Member	Presentation	Report sections	Code / demo
Saurav Singh (S20230010219)	Title, slide 1, conclusion	Abstract, 1, 6, 8, Appendix A	Server, auth & key vault, live log, Cloud View; demo part 1
Rohan Dubey (S20230010207)	Slides 2–3	2, 3, 9	Base AES-ECC scheme, Files page; demo part 2
Ujjawal (S20230010245)	Slide 4	4, 7.1–7.5	Attack Lab, analysis (entropy), experiments E1–E5; demo part 3
Aditya Raj (S20230030364)	Slide 5, results of slide 6	5, 7.6–7.8	AHE scheme, share/revoke, Results page, E6–E9; demo part 4